

Modificación de documentos

3. Modificación de documentos

Te falta la pieza final del bloque documental: modificar un documento concreto, con una particularidad que no habías tenido que resolver hasta ahora — comprobar quién tiene permiso para hacerlo.

Añadir, modificar y eliminar, en mongosh

```
// Añadir
db.nota_lectura.insertOne({ libroId: 1, autor: "ana", puntuacion: 8 })

// Modificar solo un campo concreto
db.nota_lectura.updateOne({ _id: ObjectId("...") }, { $set: { puntuacion: 9 } })

// Reemplazar el documento entero
db.nota_lectura.replaceOne({ _id: ObjectId("...") },
    { libroId: 1, autor: "ana", puntuacion: 9, comentario: "..." })

// Eliminar
db.nota_lectura.deleteOne({ _id: ObjectId("...") })
```

updateOne+\$set, replaceOne, y save()

Hay una diferencia conceptual importante entre updateOne con \$set (modifica solo los campos indicados) y replaceOne (sustituye el documento entero). save() de Spring Data MongoDB hace lo segundo — reemplaza el documento completo, no un merge parcial.

	updateOne + \$set	replaceOne	save() (Spring Data)
Qué toca	Solo los campos que indicas	Todo el documento	Todo el documento
Campos que no incluyes	Se quedan como estaban	Desaparecen	Se sobrescriben con lo que traiga el objeto Java
¿Lo usas desde Java?	No	No	Sí — es lo que ya usas

El riesgo silencioso de `save()`

`save()` no hace un merge: sustituye el documento entero por el objeto Java que le pasas. Si ese objeto llega sin todos los campos rellenos, MongoDB los sobrescribe con null sin ningún aviso — a diferencia de `$set`, que solo toca lo que le dices explícitamente. Por eso el patrón de este apartado siempre carga el documento completo antes de tocar nada.

Un ejemplo completo: control de autoría

Cualquier usuario autenticado puede publicar una nota de lectura, pero solo el autor original debería poder modificarla después. Sin esa comprobación, cualquiera con sesión iniciada podría sobrescribir la puntuación y el comentario de otra persona — hasta ahora la única barrera ha sido "estás identificado", nunca "eres el dueño de esto".

De dónde sale la identidad: Principal

```
@PostMapping
public ResponseEntity<NotaLecturaResponseDTO> create(
    @PathVariable Long libroId,
    @Valid @RequestBody NotaLecturaRequestDTO dto,
    Principal principal
) {
    NotaLecturaCreateDTO createDTO = new NotaLecturaCreateDTO(
        libroId, principal.getName(), dto.puntuacion(), dto.comentario());
    return ResponseEntity.status(HttpStatus.CREATED)
        .body(notaLecturaService.create(createDTO));
}
```

`principal.getName()` toma la identidad del usuario autenticado, inyectada por Spring Security a partir del JWT que construyes en PSP — si la petición llega autenticada, Principal te da el nombre de quien la hizo.

El patrón de control de autoría

```
public NotaLecturaResponseDTO update(
    String notaLecturaId, NotaLecturaRequestDTO dto, String usuarioActual) {
    NotaLectura notaLectura = notaLecturaRepository.findById(notaLecturaId)
        .orElseThrow(() -> new ResponseStatusException(
            HttpStatus.NOT_FOUND, "Nota de lectura no encontrada"));

    if (!notaLectura.getAutor().equals(usuarioActual)) {
        throw new ResponseStatusException(
            HttpStatus.FORBIDDEN, "Solo el autor puede modificar esta nota");
    }

    notaLectura.setPuntuacion(dto.puntuacion());
    notaLectura.setComentario(dto.comentario());
    return mapToDTO(notaLecturaRepository.save(notaLectura));
}
```

El patrón, en tres pasos

Cargar el documento por id, comprobar que el campo autor guardado coincide con `usuarioActual`, y solo entonces aplicar los cambios y guardar.

Si no coincide, 403 Forbidden — el usuario está autenticado (sabemos quién es), pero no tiene permiso para esta acción concreta sobre este recurso. Si la nota de lectura no existe, 404 Not Found, como ya conoces.

PUT /notas-lectura/{id}: el flujo de decisión

PUT /notas-lectura/{id}

¿Existe el documento?

No → 404 Not Found

Sí → continúa

¿autor == usuarioActual?

No → 403 Forbidden

Sí → Modifica y guarda → 200 OK

Por qué en ese orden, y no al revés

Primero averiguas si el recurso existe (si no, 404), y solo después comparas el autor (si no coincide, 403) — invertirlo no tendría sentido, porque no puedes comparar el autor de un documento que todavía no has cargado.

En la práctica, el autor no es la única vía de permiso

Casi cualquier aplicación real añade una segunda vía: un rol de moderación (un ADMIN) que pueda actuar igual sin ser el autor. La condición pasa de `autor == usuarioActual` a `autor == usuarioActual O esAdmin` — el rol no sustituye la comprobación de autoría, se suma a ella como una vía alternativa de permiso.

El id ya es único — ¿para qué comprobar el padre?

Si la ruta anida el recurso bajo otro (PUT /libros/{librold}/notas-lectura/{id}), el id de la nota ya basta para encontrarla. Pero como la URL afirma que esa nota pertenece a ese libro, conviene comprobarlo también: si pertenece a otro libro distinto, trátalo igual que si no existiera (404) — para esa URL, ese recurso no está.

save() y deleteById, otra vez

NotaLecturaRepository ya tiene `save()` heredado, y sirve tanto para crear un documento nuevo como para actualizar uno existente — basta con que el objeto tenga el mismo id que ya existe en la colección. No es un concepto nuevo: es exactamente lo mismo que ya viste con `JpaRepository` en el Tema 1.

Lo mismo pasa con `deleteById(id)`, heredado de `MongoRepository` igual que en `JpaRepository` — el método en sí no hay que escribirlo. Lo que sí construyes tú es el endpoint que lo expone con el mismo control de autoría o rol de administrador.

Una alternativa más avanzada

El patrón de hoy hace dos pasos en Java: cargar el documento, comprobar el autor, y solo entonces guardar. Hay una forma más directa: exigir esa misma condición dentro del propio filtro de la actualización — `db.nota_lectura.updateOne({ _id: ..., autor: usuarioActual }, { $set: {...} })`.

Si el autor no coincide, el filtro no encuentra nada que actualizar: cero documentos modificados, sin lectura previa. Es más eficiente, pero pierdes precisión en el error — sin una consulta aparte, no puedes distinguir "no existe" de "no es tuyo".

- `updateOne` con `$set` modifica campos concretos; `replaceOne` (y `save()` de Spring Data) sustituye el documento entero — y lo hace en silencio, sin avisar si el objeto Java llega con campos vacíos.
- `Principal` te da la identidad del usuario autenticado, inyectada por Spring Security a partir del JWT construido en PSP.

Ideas clave (cont.)

- El patrón de control de autoría: cargar → comprobar que el autor coincide con el usuario actual → solo entonces modificar y guardar. 403 si no coincide, 404 si no existe — en ese orden, porque no puedes comparar el autor de algo que no has cargado todavía.
- En la práctica, un rol de moderación (ADMIN) suele añadirse como segunda vía de permiso junto al autor — `autor == usuarioActual O esAdmin`, no en sustitución de la comprobación de autoría.

Ideas clave (cont.)

- Si la URL anida el recurso bajo un padre, comprueba también que el documento pertenece a ese padre, aunque su propio id ya sea único — si no coincide, 404, igual que si no existiera para esa URL.
- `save()/deleteById()` de `MongoRepository` sirven tanto para crear/actualizar como para eliminar un documento suelto, exactamente igual que `JpaRepository` — mismo concepto, distinto motor.
- Filtrar la autoría directamente en el `updateOne` es más eficiente pero pierde precisión en el error — no se practica en este módulo, pero conviene saber que existe.